C06 Collisions and Triggers

CRE132 worksheet

What you'll make

The C05 player gets three new scripts: coins to collect, a slime that turns you red when you touch it, and a jump that only works when you're standing on the ground.

You'll learn the difference between a **collision** (two solid things bump) and a **trigger** (something enters an area), how **tags** tell your code what it touched, and how **layers** and the **collision matrix** decide what physics can see. The jump uses a **raycast**: a short invisible line from the player's feet that checks for ground.

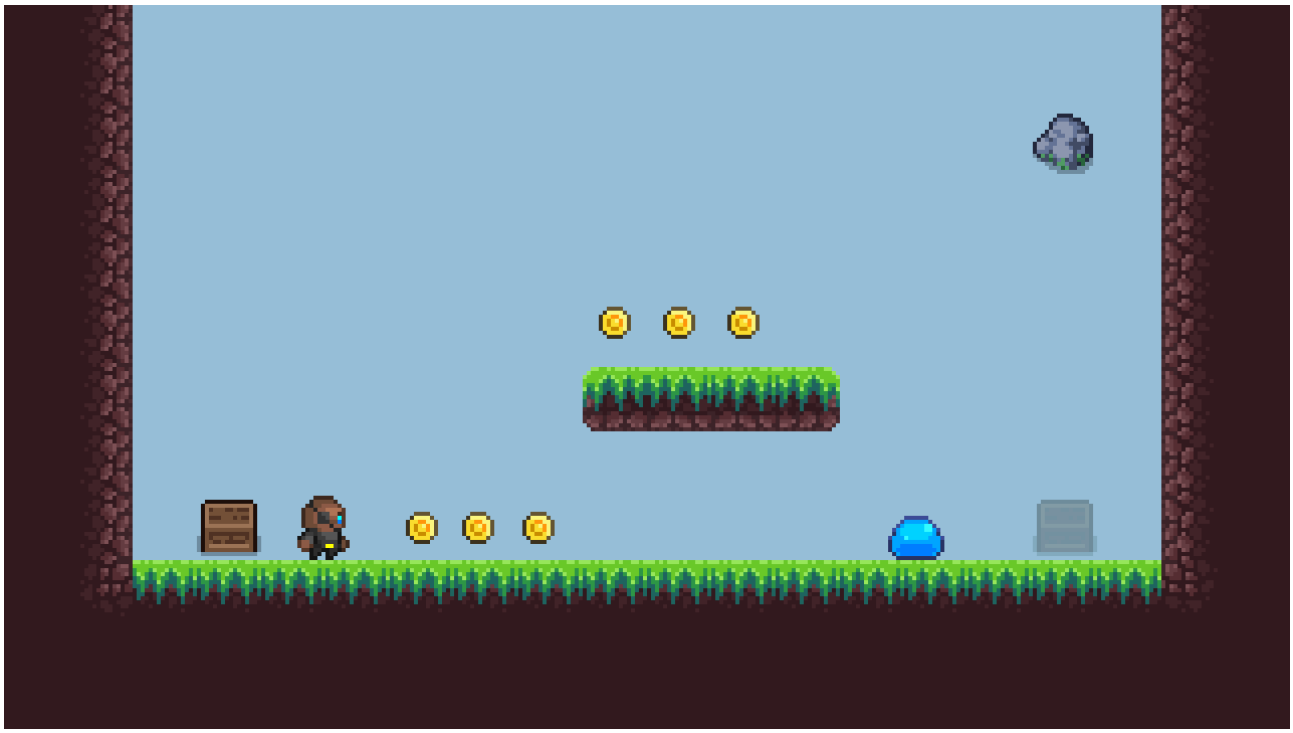


Figure 1: The finished scene: the player by a crate, two rows of coins, a floating platform, a slime and a see-through ghost block.

Before you start

1. Unzip `CRE132_C06_CollisionsAndTriggers_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_C06_CollisionsAndTriggers_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Core/C06_CollisionsAndTriggers`.
3. Press **Play**. You can walk with A/D or the arrows (C05's **PhysicsPlayerMovement** is already on the Player), but you can't jump, and the coins and slime do nothing. Press Play again to stop.

The scene has the **Ground** with a platform, six **Coins**, a **Slime**, a **Ghost Block** with a **Rock** above it, and a **Crate**.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. A trigger: Coin

Click one of the **Coins**. Its **Circle Collider 2D** has **Is Trigger** ticked. A trigger collider is not solid: things pass through it, and instead of a bump Unity calls `OnTriggerEnter2D` on the scripts of both objects.

Make a folder `Assets/_game/Scripts/C06_CollisionsAndTriggers` and create `Coin` in it:

```
// A coin the player collects by walking through it.
// The coin's collider has Is Trigger ticked, so nothing bumps into it: Unity just
// tells us when something enters it.
using UnityEngine;

public class Coin : MonoBehaviour
{
    private void OnTriggerEnter2D(Collider2D other)
    {
        // Only the player can collect coins, not crates or rocks.
        if (other.CompareTag("Player"))
        {
            Debug.Log("Coin collected!");
            Destroy(gameObject);
        }
    }
}
```

- **`OnTriggerEnter2D(Collider2D other)`** runs once when something enters the coin's trigger. `other` is the collider that came in.
- **`CompareTag("Player")`** checks the other object's **Tag**. Select the **Player**: its Tag (at the top of the Inspector) is already set to **Player**.
- **`Destroy(gameObject)`** removes the coin itself.

Add it: select all six **Coins** (inside **Coins**; click the first, Shift-click the last) and **Add Component > Coin**.

Test it: walk through the coins on the ground. Each one disappears and the Console says "Coin collected!".

2. A collision: CollisionReporter

The **Slime** has a solid collider (Is Trigger off). When two solid colliders touch, Unity calls `OnCollisionEnter2D` instead. For either message to happen, at least one of the two objects needs a **Rigidbody 2D**: here, the Player has one.

Create `CollisionReporter` in the same folder:

```
// Reports what the player bumps into, and turns the player red while touching an
// enemy.
// Collisions happen between two solid colliders (Is Trigger off), and at least one
// needs a Rigidbody2D.
using UnityEngine;

public class CollisionReporter : MonoBehaviour
{
```

```
    private SpriteRenderer spriteRenderer;

    private void Awake()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
    }

    // Called once, on the first physics step where the two colliders touch.
    private void OnCollisionEnter2D(Collision2D collision)
    {
        Debug.Log("Bumped into " + collision.gameObject.name);

        // Tags let us ask "what kind of thing is this?" without checking names.
        if (collision.gameObject.CompareTag("Enemy"))
        {
            Debug.Log("Ouch! That was an enemy.");
            spriteRenderer.color = Color.red;
        }
    }

    // Called once, when the two colliders stop touching.
    private void OnCollisionExit2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Enemy"))
        {
            spriteRenderer.color = Color.white;
        }
    }
}
```

- **A collision gives you a `Collision2D`**, not a `Collider2D`. `collision.gameObject` is the thing you hit.
- **Enter and Exit:** Enter turns the player red when it touches an enemy, Exit turns it white again when it steps away. There are `Stay` versions too, which run every physics step while touching.
- **Tags, not names:** the Slime's Tag is **Enemy**. Checking the tag means every enemy works, whatever it's called. To make a new tag, click **Tag > Add Tag...** in the Inspector.

Add it: select the **Player** and **Add Component > CollisionReporter**.

Test it: walk into the Slime. The player turns red and the Console says "Ouch! That was an enemy." Step away and it turns white again. Walking into the Crate only logs "Bumped into Crate".

3. Layers and the collision matrix

Tags answer "what is it?" in your code. **Layers** decide what physics (and cameras) can see. Click these objects and look at **Layer** at the top of the Inspector:

Object	Layer
Ground	Ground
Player	Player
Ghost Block	Ghost
Crate, Rock	Default

Now open **Edit > Project Settings > Physics 2D** and find the **Layer Collision Matrix**. Each tick means "these two layers collide". The box where the **Player** row meets the **Ghost** column is **unticked**.

Test it: walk into the Ghost Block. The player walks straight through it, but the Rock above it lands on it and stays there, because Default and Ghost still collide.

4. Jumping from the ground: GroundCheckJump

Create `GroundCheckJump` in the same folder:

```
// Lets the player jump with Space, but only while standing on the ground.
// A short raycast from the feet downwards tells us whether there is ground below.
// Works alongside PhysicsPlayerMovement (C05), which handles walking left and right.
using UnityEngine;

public class GroundCheckJump : MonoBehaviour
{
    [SerializeField] private float jumpSpeed = 9f;
    [SerializeField] private float rayLength = 0.15f;

    // Only colliders on these layers count as ground. This also stops the ray
    // from hitting the player's own collider.
    [SerializeField] private LayerMask groundLayers;

    private Rigidbody2D body;
    private bool jumpPressed;
    private bool isGrounded;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
    }

    private void Update()
    {
        // Remember the key press until FixedUpdate runs.
        if (Input.GetKeyDown(KeyCode.Space))
        {
            jumpPressed = true;
        }
    }

    private void FixedUpdate()
    {
        // The player's pivot is at its feet, so start the ray a little above the
        // feet.
        Vector2 rayStart = new Vector2(transform.position.x,
            transform.position.y + 0.05f);
        RaycastHit2D hit = Physics2D.Raycast(rayStart, Vector2.down, rayLength,
            groundLayers);

        if (hit.collider != null)
        {
            isGrounded = true;
        }
        else
        {
            isGrounded = false;
        }

        // Draw the ray in the Scene view: green when grounded, red in the air.
        Color rayColor = Color.red;
        if (isGrounded)
```

```
        {
            rayColor = Color.green;
        }
        Debug.DrawRay(rayStart, Vector2.down * rayLength, rayColor);

        if (jumpPressed && isGrounded)
        {
            body.linearVelocity = new Vector2(body.linearVelocity.x, jumpSpeed);
        }

        // Forget the press either way, so a press in mid-air doesn't jump on
        // landing.
        jumpPressed = false;
    }
}
```

- **`Physics2D.Raycast`** shoots an invisible line from `rayStart`, straight down, `rayLength` units long, and returns what it hit first. If `hit.collider` is not `null`, there is ground under the feet.
- **The `LayerMask`** makes the ray only see the layers you choose. Without it, the ray would hit the player's own collider and you could jump in mid-air.
- **`jumpPressed`** carries the key press from `Update` (where no press is missed) to `FixedUpdate` (where the Rigidbody is changed), as in C05.
- **The jump only sets the `y` velocity**, keeping the `x` that PhysicsPlayerMovement set. Both scripts share one Rigidbody 2D.

Add it: select the **Player** and **Add Component > GroundCheckJump**. Leave **Jump Speed** at **9** and **Ray Length** at **0.15**. Set **Ground Layers** to **Ground** only (click the drop-down and tick **Ground**).

Test it: press Play and jump with **Space**. Jump onto the platform and collect the coins above it. With the Scene view open beside the Game view, watch the short ray at the player's feet: green on the ground, red in the air.

Now see why the mask matters: set **Ground Layers** to **Everything** and play. Press Space in mid-air: you can jump again and again, because the ray hits the player's own collider. Set it back to **Ground**.

5. The crate

Stand on the **Crate** and press Space. Nothing happens: the Crate is on the **Default** layer, and only **Ground** counts as ground. Stop, set the Crate's **Layer** to **Ground** and try again: now you can jump off it. Set it back to **Default** afterwards if you want the Finished behaviour.

Check your work

- ☐ Walking or jumping through a coin collects it and logs "Coin collected!".
- ☐ Touching the Slime turns the player red; stepping away turns it white.
- ☐ The player walks through the Ghost Block, but the Rock sits on it.
- ☐ Space jumps only from the ground and the platform, never in mid-air or off the crate.
- ☐ Compare with `CRE132_C06_CollisionsAndTriggers_Finished` if something behaves differently.

Key points

- **Collision = a solid bump** (`OnCollisionEnter2D`). **Trigger = something entered this area** (`OnTriggerEnter2D`). At least one of the two needs a Rigidbody 2D.
- **Messages arrive on both objects.** Put the script on whichever one should react: the coin reacts to the player, the player reacts to the slime.
- **Tags answer "what is it?" in code. Layers decide what physics and raycasts see.**
- **A raycast needs a layer mask**, or it can hit the object it starts inside.

If something goes wrong

- **Coins don't disappear:** the Player's Tag isn't **Player** (check the spelling and capital P), or the coin's collider isn't a trigger.
- **The slime does nothing:** its Tag isn't **Enemy**, or CollisionReporter is on the slime instead of the Player.
- **NullReferenceException in CollisionReporter:** the script is on an object with no Sprite Renderer.
- **Space never jumps: Ground Layers** is set to **Nothing**. Tick **Ground**.
- **You can jump in mid-air: Ground Layers** includes the **Player** layer (or Everything).
- **The player gets stuck walking into the Ghost Block:** the Player × Ghost box in the collision matrix is ticked.

Try it

1. Add a coin counter: an `int` in a new script on the player, increased from `Coin`. (S02 does this properly with a GameManager.)
2. Make the Slime a trigger instead. What changes about the red flash, and why?
3. Put the Rock on the **Player** layer. What happens to it at the Ghost Block?